Presentation Study Guide

3D Logo "DO" — Daniel Oni — CS 5382

1. What the Project Is

This program shows my initials **"DO"** sitting on top of a pedestal, all in 3D. The pedestal is a cylinder that's wider at the bottom and narrower at the top (like a trophy base). The user can tilt and turn the scene with sliders, spin the letters on the pedestal, switch between 4 color themes, and toggle confetti.

Files

- **CG_Program1_OniDaniel.html** — the shaders (GLSL code that runs on the GPU), the HTML controls (sliders, buttons, dropdown), and the CSS styling.
- **CG_Program1_OniDaniel.js** — all the JavaScript: builds the shapes, talks to the GPU, handles user input, and runs the animation loop.

How it runs (big picture)

When you open the HTML file, **window.onload** fires and does everything in order: sets up the GPU program, builds all the shapes as numbers (vertices), sends them to the GPU once, then starts a loop that redraws the scene every frame. The loop only changes rotation angles — it never re-sends the shape data.

2. How Each Shape Is Built

The Pedestal

It's a cylinder where the bottom circle is wider than the top circle. Both circles are made using **polar coordinates**:

```
x = radius * cos(angle)
z = radius * sin(angle)
```

I go around the circle in 36 steps (every 10 degrees). At each step, I grab one point on the bottom circle and one on the top circle, then connect them with triangles. I also fill in the top and bottom with triangles from the center point outward (like pie slices). That gives a completely solid, closed shape — no holes anywhere.

Property	Value
Bottom radius	0.48
Top radius	0.35
Bottom y-position	-0.65 (sits on XZ-plane)
Top y-position	-0.15
Segments	36 (smooth circle)

Letter D

Made of two pieces glued together:

- **Vertical bar** — a simple 3D box (6 faces, each made of 2 triangles).
- **Curved part** — a half-ring (semicircle) using polar coordinates from -90 degrees to +90 degrees. It has front, back, outer, inner walls, and edge caps to seal the ends.

The curve's outer radius is exactly half the bar's height. This makes the curve meet the bar perfectly at the top and bottom — no gaps or stubs.

Letter O

A full ring (0 to 360 degrees) using the same ring-building function as the D's curve. Since it goes all the way around, it closes on itself — no edge caps needed.

Why centering matters

The letters spin around the Y-axis (the vertical line at x = 0). If the letters aren't centered at x = 0, they orbit like a planet instead of spinning in place. I calculated the combined width of D + O, found the midpoint, and shifted everything so the center lands exactly at x = 0. This was a bug I had to fix.

Presentation tip: This is a great thing to talk about in your presentation — a real challenge you solved with math.

3. How Rotation Works

The vertex shader builds 4x4 rotation matrices (just like the class examples from Ex_02_4). There are 4 rotation uniforms:

Uniform	What it does	Controlled by
uThetaX	Tilts the view up/down	X-axis slider
uThetaY	Turns the view left/right	Y-axis slider
uThetaZ	Unused (always 0)	—
uExtraThetaY	Spins letters on pedestal	Auto-rotate button

The final vertex position is calculated as:

```
gl_Position = Rx * Ry * Rz * RyExtra * aPosition;
```

RyExtra is applied first (spins the letters), then the scene rotation (Rx, Ry) changes your viewing angle. The key trick: when drawing the pedestal, I set uExtraThetaY to 0 so it doesn't spin. When drawing the letters, I set it to the current auto-rotate angle. Same shader, different uniform values = different behavior.

Column-major order

GLSL fills matrices column by column, not row by row. So when you write mat4(...) the first 4 numbers are column 0. This is just how GLSL works — same as the class examples.

4. The 3 Interaction Types

Type	What	How it works
Sliders (2x)	Tilt + Turn the 3D view	oninput converts the slider's degree value to radians and saves it; the render loop
Buttons (2x)	Toggle auto-rotate + confetti	onclick flips a true/false flag; render loop checks the flag to decide what to draw
Dropdown (1x)	Switch color theme	onchange rebuilds all vertices with new colors and re-uploads to GPU (this is the

5. Questions People Might Ask You

Q1: How did you make the pedestal wider at the bottom?

Two circles with different radii, both made with polar coordinates. Connect them with triangles. The bottom one uses radius 0.48, the top uses 0.35.

Q2: How do the letters spin but the pedestal stays still?

I use an extra uniform (uExtraThetaY). I set it to 0 when drawing the pedestal, and set it to the current spin angle when drawing the letters. Same shader, different uniform value.

Q3: Why do you rebuild vertices when the theme changes?

Colors are stored inside the vertex data (each vertex has x,y,z,w,r,g,b). New theme = new colors = need to send new vertex data to the GPU. This only happens when you pick a new theme, NOT every frame.

Q4: Are all the surfaces closed / solid?

Yes. The pedestal has side walls + top cap + bottom cap. The letters have front, back, outer wall, inner wall, and edge caps. You can rotate to any angle and never see through the objects.

Q5: What was the hardest part?

Centering the letters for auto-rotate. They were off-center by about 0.12 in x, which made them orbit instead of spin. I calculated the exact combined bounding box center and shifted everything to fix it.

Q6: Why TRIANGLES instead of TRIANGLE_STRIP?

With plain TRIANGLES, everything can share one buffer and I just change the start index for each draw call. Strips would need separate calls or degenerate triangles to connect different objects.

Q7: How does depth testing help?

gl.enable(gl.DEPTH_TEST) uses a z-buffer so front faces hide back faces. Without it, whatever you draw last appears on top, even if it's behind something.

Q8: How many vertices total?

About 1,800. Pedestal ~432, Letter D ~660, Letter O ~576, Confetti ~120. All stored in one buffer, drawn with separate drawArrays calls.

6. Requirements Checklist

Quick reference to confirm everything is covered:

Visual (30 pts)

- Two+ 3D objects with width/height/depth — **pedestal + letters**
- Solid closed surfaces with triangles — **all faces covered**
- Pedestal is a cylinder, base parallel to XZ-plane — **y = -0.65 base**
- Pedestal has different top/bottom radii — **0.48 bottom, 0.35 top**
- Initials on pedestal — **"DO" sits on top**

Interaction (10 pts)

- 3 different interaction types — **sliders, buttons, dropdown**
- User can change 3D view — **tilt + turn sliders**
- At least 2 rotation changes — **X-axis + Y-axis**

Implementation (20 pts)

- Uses class code patterns — **initShaders(), mat4 rotation, buffer setup from Ex_02_4**
- Separate HTML and JS files — **yes**
- Vertices NOT in render loop — **uploaded once in init, only on theme change**
- All vertices procedural — **no imports, all generated by JS functions**
- Polar coordinates for pedestal — **x = r*cos(theta), z = r*sin(theta)**
- Vertices in clipping cube [-1, 1] — **max extent is 0.48, well within bounds**

Quality Bonuses (10 pts)

- Confetti effect toggle
- 4 color themes with styled dropdown
- Angle-based shading on pedestal for depth
- Clean dark UI with hover/active button styles
- Auto-rotate on by default (animated variation)